

Text Mining in R – Ngrams and Log Odds

Gosse Bouma, Defne Abur

Feb, 2025

Goal

The goal of this assignment is:

- to learn how to collect statistics for combinations of 2 or more words (n-grams)
- to inspect some of the results for negation
- to experiment with the log-odds method for detecting keywords

Book Review Dataset Again

We will use the same type of data as for the previous assignment; i.e., book reviews from the the Amazon Review dataset

We will use the same type of data as for the previous assignment. For this assignment, however, the samples are 10 times larger (each file contains 5000 reviews). This time, please pick the sample that corresponds to the first number of your student number (i.e. student s123456 will work with pos1.txt and neg1.txt).

All data can be found on Google drive

```
# Load data
# My snumber is s5788668, so I will use pos5.txt and neg5.txt
pos <- read_lines(here("data", "raw", "corpusComparision", "pos5.txt"))
neg <- read_lines(here("data", "raw", "corpusComparision", "neg5.txt"))
head(pos, 2)
```

```
## [1] "I love D. Gabaldon's Outlander series, but I find myself struggling to be interested in some pa
## [2] "This book was great as a intro into programming with C++. It covers the basics really well and I
```

```
head(neg, 2)
```

```
## [1] "Before reading A Christmas Carol, I automatically gave the book the highest regard probably bec
## [2] "I have enjoyed many of Anne Tyler's books, particularly, \"Dinner at the Homesick Restaurant,\"
```

This goal of this assignment is to investigate:

- differences between negative vs positive reviews
- bigrams
- negation
- aspects

- collocations

In sentiment analysis, aspects refer to a particular aspect of a product on which a sentiment is expressed. Concretely, a book review may say “the plot was great, but the writing style was terrible”; “plot” and “writing style” are aspects.

A: Tokenizing N-grams

(2 points)

The `readr` package provides a function `read_lines()` that can be used to read arbitrary text and store the result as a vector of strings. 1. Use this to read the two files. 2. Next, create a `tibble` dataframe for the data of the positive review. 3. Create a `tibble` dataframe for the data of the negative review. 4. Tokenize the review text into bigrams, as described in chapter 4.1 of Text mining in R.

```
# Create tibble dataframes
pos_df <- tibble(text = pos)
neg_df <- tibble(text = neg)

# Tokenize the review text into bigrams
pos_bigram <- pos_df |>
  unnest_tokens(bigram, text, token = "ngrams", n = 2) |>
  filter(!is.na(bigram)) # I understand but disapprove it
# Adding <s></s> can solve the problem, no need to sacrifice semantic info

neg_bigram <- neg_df |>
  unnest_tokens(bigram, text, token = "ngrams", n = 2) |>
  filter(!is.na(bigram))
```

B: Splitting N-grams

(3 points)

1. Split the bigram column into two columns `word1` and `word2`, as described in chapter 4.1 (e.g., using the `separate()` function). Do not apply stopwords filtering.
2. Determine the 5 most frequent words preceding ‘style’ in positive reviews (if there are ties, just report the first 5!)
3. Determine the 5 most frequent words preceding ‘style’ in negative reviews
4. Repeat the above two steps for the word ‘characters’ (for both positive and negative reviews)
5. Do you see interesting differences between positive and negative reviews for these two words? Comment on your findings.
6. How often do positive and negative reviews mention each of these words (i.e., what is the relative frequency of these words)? Comment on your findings.

```
# To improve the readability of the code, I will keep it in one chunk, with
# comments for each step

# B1: Split the bigram column into two columns word1 and word2
pos_bigrams_separated <- pos_bigram %>%
  separate(bigram, c("word1", "word2"), sep = " ")

neg_bigrams_separated <- neg_bigram %>%
```

```

separate(bigram, c("word1", "word2"), sep = " ")

# B2: 5 most frequent words preceding 'style' in positive reviews
pos_style <- pos_bigrams_separated %>%
  filter(word2 == "style") %>%
  count(word1, sort = TRUE) %>%
  head(5)

# B3: 5 most frequent words preceding 'style' in negative reviews
neg_style <- neg_bigrams_separated %>%
  filter(word2 == "style") %>%
  count(word1, sort = TRUE) %>%
  head(5)

# B4: 5 most frequent words preceding 'characters' in pos & neg reviews
pos_characters <- pos_bigrams_separated %>%
  filter(word2 == "characters") %>%
  count(word1, sort = TRUE) %>%
  head(5)

neg_characters <- neg_bigrams_separated %>%
  filter(word2 == "characters") %>%
  count(word1, sort = TRUE) %>%
  head(5)

# Print results
print("Words preceding 'style' in positive reviews:")

```

```
## [1] "Words preceding 'style' in positive reviews:"
```

```
print(pos_style)
```

```
## # A tibble: 5 x 2
##   word1      n
##   <chr>  <int>
## 1 writing    75
## 2 the       28
## 3 his       16
## 4 a         12
## 5 and       11
```

```
print("Words preceding 'style' in negative reviews:")
```

```
## [1] "Words preceding 'style' in negative reviews:"
```

```
print(neg_style)
```

```
## # A tibble: 5 x 2
##   word1      n
##   <chr>  <int>
## 1 writing   136
```

```
## 2 the      50
## 3 his      18
## 4 a        6
## 5 and      6
```

```
print("Words preceding 'characters' in positive reviews:")
```

```
## [1] "Words preceding 'characters' in positive reviews:"
```

```
print(pos_characters)
```

```
## # A tibble: 5 x 2
##   word1      n
##   <chr> <int>
## 1 the    332
## 2 main   48
## 3 of     38
## 4 his    29
## 5 other  29
```

```
print("Words preceding 'characters' in negative reviews:")
```

```
## [1] "Words preceding 'characters' in negative reviews:"
```

```
print(neg_characters)
```

```
## # A tibble: 5 x 2
##   word1      n
##   <chr> <int>
## 1 the    557
## 2 main   71
## 3 these  38
## 4 of     36
## 5 his    32
```

```
# B6: Relative frequency of 'style' and 'characters' in both pos & neg reviews
```

```
style_freq <- bind_rows(
  pos_bigrams_separated %>%
    filter(word2 == "style") %>%
    summarize(type = "positive", count = n(),
              total = nrow(pos_bigrams_separated),
              rel_freq = count/total),
  neg_bigrams_separated %>%
    filter(word2 == "style") %>%
    summarize(type = "negative", count = n(),
              total = nrow(neg_bigrams_separated),
              rel_freq = count/total)
)
```

```
characters_freq <- bind_rows(
```

```

pos_bigrams_separated %>%
  filter(word2 == "characters") %>%
  summarize(type = "positive", count = n(),
            total = nrow(pos_bigrams_separated),
            rel_freq = count/total),
neg_bigrams_separated %>%
  filter(word2 == "characters") %>%
  summarize(type = "negative", count = n(),
            total = nrow(neg_bigrams_separated),
            rel_freq = count/total)
)

print("Relative frequency of 'style':")

```

```
## [1] "Relative frequency of 'style':"
```

```
print(style_freq)
```

```
## # A tibble: 2 x 4
##   type      count total rel_freq
##   <chr>    <int> <int>   <dbl>
## 1 positive   357 869234 0.000411
## 2 negative   419 910246 0.000460
```

```
print("Relative frequency of 'characters':")
```

```
## [1] "Relative frequency of 'characters':"
```

```
print(characters_freq)
```

```
## # A tibble: 2 x 4
##   type      count total rel_freq
##   <chr>    <int> <int>   <dbl>
## 1 positive   932 869234 0.00107
## 2 negative  1294 910246 0.00142
```

Answer: B5: Differences Between Positive and Negative Reviews

For “style,” both positive and negative reviews primarily discuss “writing style,” but negative reviews mention it almost twice (136 vs. 75 occurrences). Interestingly, the top 5 preceding words remain identical across both review types, suggesting very different, even opposite, opinions can be discussed and conveyed by similar language usage.

For “characters,” negative reviews reference them more specifically, using “these characters” more frequently, while positive reviews uniquely mention “other characters.” This suggests negative reviews might focus on criticizing specific character issues, while positive reviews tend to compare characters or discuss their relationships.

B6: Relative Frequency Insights

Style appears *slightly* more often in negative reviews (0.00046 vs. 0.00041), but the difference is nominal. However, characters are mentioned substantially more frequently in negative reviews (0.00142 vs. 0.00107) – about 33% higher. This suggests that character flaws may be a stronger driver of negative reviews than stylistic issues, with poorly developed or unlikeable characters being a common criticism when readers dislike a book.

C: Negation and sentiment

(3 points)

Section 4.1.3 discusses how naive sentiment classification might get the sentiment wrong of phrases such as ‘no fun’. Whereas ‘fun’ has a positive score, this bigram as a whole should be considered negative (or at least should not contribute to the positive score).

Again, do not apply stop word filtering. In this question we will focus on bigrams with “not” as the first word.

1. Determine the 5 most frequent words following ‘not’ in positive reviews? this means the format of [not, xxx]
2. Annotate word2 with sentiment scores from the AFINN lexicon as in the previous assignment (i.e., add a column with sentiment scores for the second word).
3. Create a bar plot as in Figure 4.2 of the textbook.
4. The plot should show for all the “not . . .” bigrams: Which 10 words contribute most in the positive direction? Which 10 words contribute the most in the negative direction? (Contribution is computed as frequency times sentiment value of the second word).
5. Repeat steps 1-3 for the negative reviews.
6. Do you see interesting differences between positive and negative reviews? Comment on your findings.

```
# C1: Determine the 5 most frequent words following 'not' in positive reviews
pos_not <- pos_bigrams_separated %>%
  filter(word1 == "not") %>%
  count(word2, sort = TRUE) %>%
  head(5)
```

```
print("Most frequent words following 'not' in positive reviews:")
```

```
## [1] "Most frequent words following 'not' in positive reviews:"
```

```
print(pos_not)
```

```
## # A tibble: 5 x 2
##   word2      n
##   <chr> <int>
## 1 a      290
## 2 only   213
## 3 the    178
## 4 be     156
## 5 to     156
```

```
# C2: Annotate word2 with sentiment scores from AFINN lexicon
AFINN <- get_sentiments("afinn")
```

```
pos_not_sentiment <- pos_bigrams_separated %>%
  filter(word1 == "not") %>%
  inner_join(AFINN, by = c(word2 = "word")) %>%
  count(word2, value, sort = TRUE)
```

```
# C3 & C4: Bar plot
```

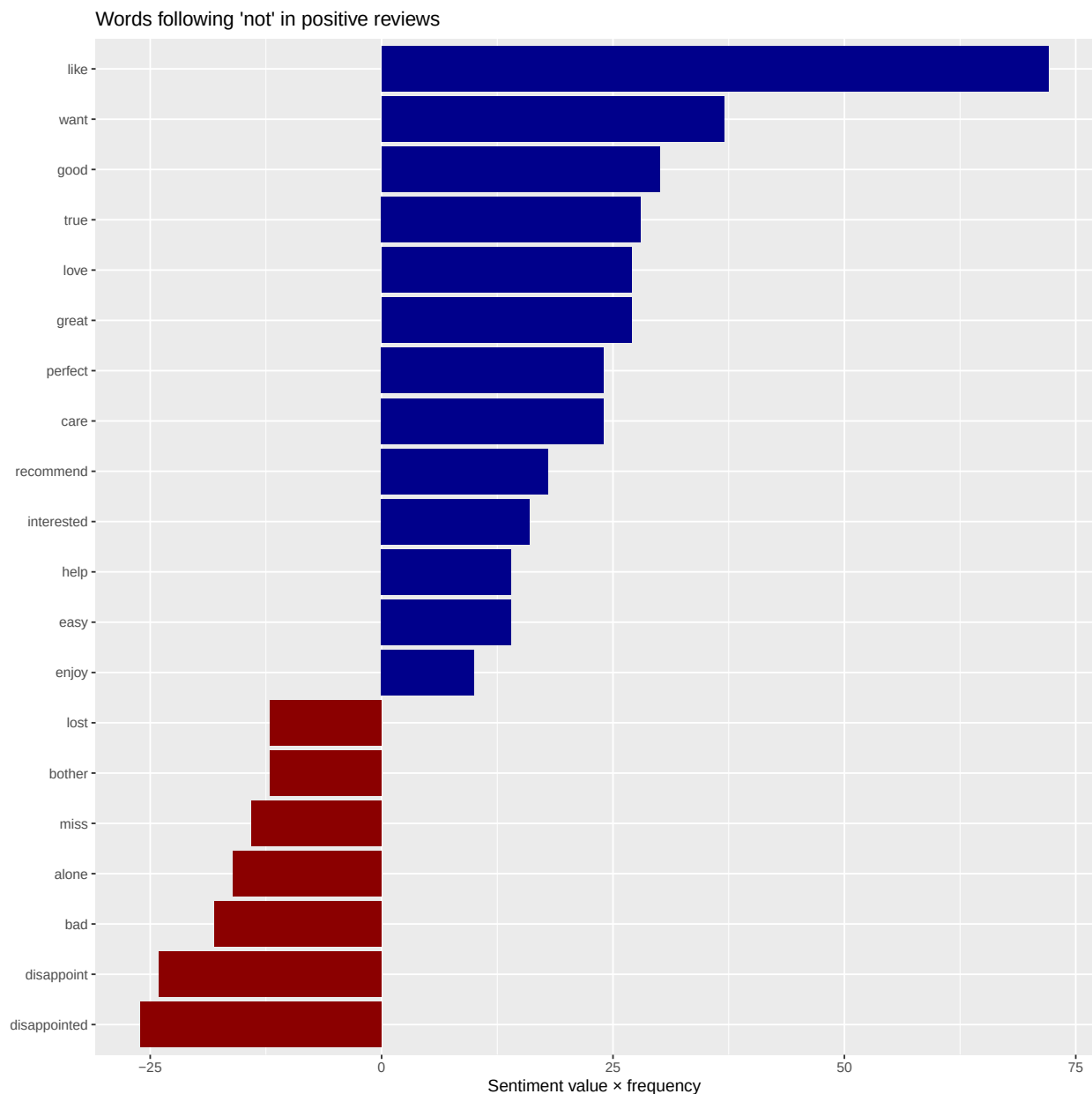
```
pos_contribution <- pos_not_sentiment %>%
```

```

mutate(contribution = n * value) %>% # chapter 2
arrange(desc(abs(contribution))) %>%
head(20) %>% # Get top 20 words by contribution magnitude
mutate(word2 = reorder(word2, contribution)) # Reorder for visualisation

# Visualisation for positive reviews
ggplot(pos_contribution, aes(contribution, word2, fill = contribution > 0)) +
  geom_col(show.legend = FALSE) +
  labs(x = "Sentiment value × frequency",
       y = NULL,
       title = "Words following 'not' in positive reviews") +
  scale_fill_manual(values = c("darkred", "darkblue"))

```



```
# C5: Repeat for negative reviews
neg_not <- neg_bigrams_separated %>%
  filter(word1 == "not") %>%
  count(word2, sort = TRUE) %>%
  head(5)

print("Most frequent words following 'not' in negative reviews:")
```

```
## [1] "Most frequent words following 'not' in negative reviews:"
```

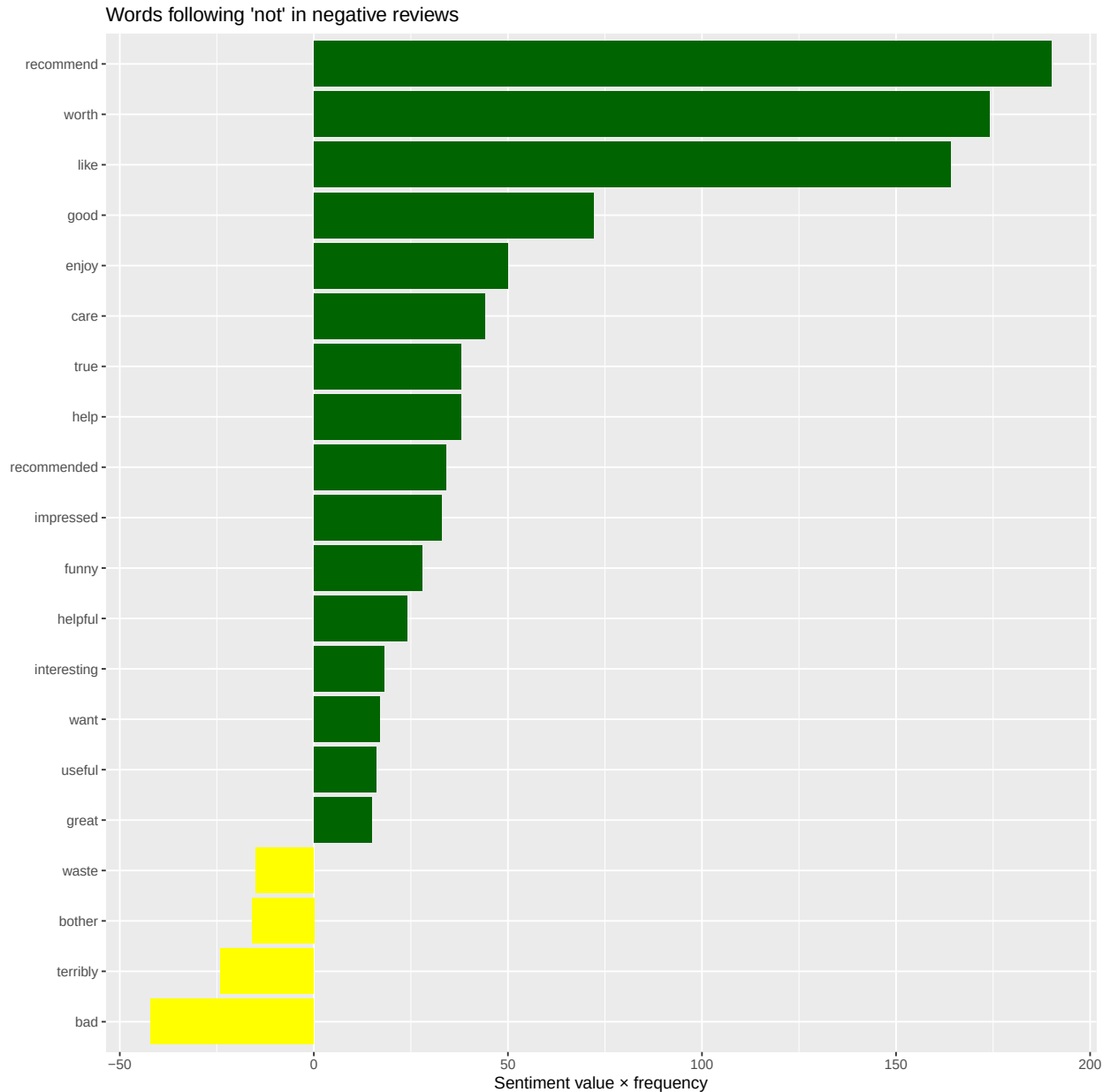
```
print(neg_not)
```

```
## # A tibble: 5 x 2
##   word2      n
##   <chr> <int>
## 1 a      448
## 2 the    276
## 3 to     187
## 4 only   141
## 5 be     138
```

```
neg_not_sentiment <- neg_bigrams_separated %>%
  filter(word1 == "not") %>%
  inner_join(AFINN, by = c(word2 = "word")) %>%
  count(word2, value, sort = TRUE)

neg_contribution <- neg_not_sentiment %>%
  mutate(contribution = n * value) %>%
  arrange(desc(abs(contribution))) %>%
  head(20) %>%
  mutate(word2 = reorder(word2, contribution))

# Visualisation for negative reviews
ggplot(neg_contribution, aes(contribution, word2, fill = contribution > 0)) +
  geom_col(show.legend = FALSE) +
  labs(x = "Sentiment value × frequency",
       y = NULL,
       title = "Words following 'not' in negative reviews") +
  scale_fill_manual(values = c("yellow", "darkgreen"))
```

Answer: C6: Differences Between Positive and Negative Reviews In negative reviews, the clear pattern of negating positive qualities can be observed: “not recommend” and “not worth” appear frequently with high contribution. As a comparison, positive reviews show more nuanced negation patterns. While they do negate negative qualities, for example, “not disappointed”, they use negation more moderately overall. The lower contribution magnitudes suggest negation plays a less critical role in positive assessments.

D: Log Odds

Here we will use the implementation of the log-odds ratio method for finding keywords as implemented in the tidylo library (<https://github.com/juliasilge/tidylo>)

The task is to compare keywords for the novels of two authors from the gutenber text collection

1. Select two authors from the gutenber collection, where the first letter of the last name of at least one of the authors corresponds to the first letter of your last name. Think of two authors that differ in for instance, style, genre, period, or nationality. At least one of the authors' name should start with "Z". Zane Grey was an American author and dentist. He is known for his popular adventure novels and stories associated with the Western genre. Émile Zola was a French novelist, journalist, playwright. More than half of Zola's novels were part of the twenty-volume Les Rougon-Macquart cycle, which details the history of a single family under the reign of Napoléon III. – quoted from Wikipedia
2. Download 3-5 texts for both authors (make sure to include the author name, will need later). Tokenize the texts and count the words per author.
3. If you have not done so already, combine the two datasets.
4. Compute the log odds for the word counts per author (see the github page of the tidylo library for an example).
5. Visualise the top-20 highest ranked words for each author in a bar plot (following the example if tidylo).
6. What are the most important differences in keywords between your authors? Comment on your findings.

```
# D1: Select authors
# Get metadata
books <- gutenber_metadata

# Filter for authors with last name starting with 'Z' and English language works
filtered_books <- books %>%
  filter(
    str_detect(author, "Z"), # Author's last name starts with Z
    language == "en",       # English language works
    has_text == TRUE        # Only books with available text
  )

# Show the first five authors to choose from
head(filtered_books, 30)
```

```
## # A tibble: 30 x 8
##   gutenber_id title      author gutenber_author_id language gutenber_bookshelf
##   <int> <chr>      <chr>          <int> <chr>      <chr>
## 1      338 Old Ind~ Zitka~          188 en      "Native America"
## 2      385 The Red~ Grey,~          212 en      "Western"
## 3      502 Desert ~ Grey,~          212 en      "Western"
## 4     1027 The Lon~ Grey,~          212 en      "Western/Bestselle~
## 5     1239 The Spi~ Grey,~          212 en      "Western"
## 6     1261 Betty Z~ Grey,~          212 en      "Western"
## 7     1262 The Her~ Grey,~          212 en      "Western"
## 8     1300 Riders ~ Grey,~          212 en      "Western"
## 9     1445 Aeropla~ Zerbe~          635 en      ""
## 10    1881 The Cal~ Grey,~          212 en      "Western"
## # i 20 more rows
## # i 2 more variables: rights <chr>, has_text <lgl>
```

I choose the authors “Grey, Zane” and “Zola, Émile” for this task. I will download 3 texts for each author.

```
# D2: Download texts
# Download Zane Grey books
# Zane Grey books
grey_book1 <- gutenber_download(502, meta_fields = "author")
```

```

grey_book2 <- gutenbergs_download(2057, meta_fields = "author")
grey_book3 <- gutenbergs_download(2066, meta_fields = "author")

# Combine Grey books
grey_books <- bind_rows(grey_book1, grey_book2, grey_book3)

# Download Émile Zola books
zola_ids <- c(8511, 8512, 8513)
zola_books <- gutenbergs_download(zola_ids, meta_fields = "author")
# For some reason Zane's books do not support this way
# I have to download each book manually and bind them together

# Follow the tidy text format & word count
grey_words <- grey_books %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words) %>% # Remove stopwords
  count(author, word, sort = TRUE)

zola_words <- zola_books %>%
  unnest_tokens(word, text) %>%
  anti_join(stop_words) %>% # Remove stopwords
  count(author, word, sort = TRUE)

# D3: Combine the data
all_words <- bind_rows(grey_words, zola_words)

# D4: Calculate log odds
word_log_odds <- all_words %>%
  bind_log_odds(author, word, n) # must be tidy data

# D5: Visualise top 20 words for each author
word_log_odds %>%
  group_by(author) %>%
  slice_max(log_odds_weighted, n = 20) %>% #select the top 20 words with highest log_odds_weighted value
  # already weighted, dont need to do anything, just use
  ungroup() %>%
  # IMPORTANT!! Reorder the words within each author
  #because the same word might appear for both authors
  # but need different positions in each facet
  mutate(word = reorder_within(word, log_odds_weighted, author)) %>%
  # Visualisation
  ggplot(aes(log_odds_weighted, word, fill = author)) +
  geom_col(show.legend = FALSE) +
  # Split the plot into separate panels (facets) for each author
  facet_wrap(~author, scales = "free_y") +
  scale_y_reordered() +
  labs(title = "Most Distinctive Words in Zane vs. Zola",
       x = "Log odds ratio",
       y = NULL)

```

Most Distinctive Words in Zane vs. Zola

